BOOLEAN RETRIEVAL MODEL:
PRAC 1

```
plays = {
    "Anthony and Cleopatra": "Anthony is there, Brutus is Caeser is with
Cleopatra mercy worser.",
    "Julius Ceaser": "Anthony is there, Brutus is Caeser is but Calpurnia is.",
    "The Tempest": "mercy worser",
    "Ham let": "Caeser and Brutus are present with mercy and worser",
    "Othello": "Caeser is present with mercy and worser",
    "Macbeth": "Anthony is there, Caeser, mercy."
}

words = ["Anthony", "Brutus", "Caeser", "Calpurnia", "Cleopatra", "mercy",
"worser"]

# Build vector matrix: rows = words, cols = plays
vector_matrix = [[0 for _ in range(len(plays))] for _ in range(len(words))]

text_list = list(plays.values())

for i in range(len(words)):
    for j in range(len(text_list)):
        if words[i] in text_list[j]:
            vector_matrix[i][j] = 1
        else:
            vector_matrix[i][j] = 0

for row in vector_matrix:
```

```
    print(row)

# Convert each word's vector to an integer
vector_dict = {}
for idx in range(len(vector_matrix)):
    bits = "".join(str(digit) for digit in vector_matrix[idx])
    vector_dict[words[idx]] = int(bits, 2)

print(vector_dict)

condition = input("Enter your condition: ")
condition1 = condition

# Replace words in condition with their binary numbers
for w in words:
    condition1 = condition1.replace(w, str(vector_dict[w]))

# Optional: support 'and'/'or'
condition1 = condition1.replace(" and ", " & ")
condition1 = condition1.replace(" or ", " | ")

# Replace "not" with "~"
condition1 = condition1.replace("not", "~")

result = eval(condition1)
str1 = bin(result).replace("ob", "").zfill(len(plays))

print("The plays which satisfy the condition", condition, "are")
print(str1)
```

```
play_names = list(plays.keys())
for idx, bit in enumerate(str1):
    if bit == "1":
        print(play_names[idx])
```

SIMPLE WALA

```
plays = {
    "Anthony and Cleopatra": "Anthony is there, Brutus is Caeser is with
Cleopatra mercy worser.",
    "Julius Ceaser": "Anthony is there, Brutus is Caeser is but Calpurnia is.",
    "The Tempest": "mercy worser",
    "Hamlet": "Caesar and Brutus are present with mercy and worser",
    "Othello": "Caesar is present with mercy and worser",
    "Macbeth": "Anthony is there, Caesar, mercy."
}

words =
["Anthony","Brutus","Caeser","Calpurnia","Cleopatra","mercy","worser"]

# Step 1: Create binary vectors
vectors = {}

for word in words:
    binary = ""
    for play in plays.values():
        if word in play:
            binary += "1"
        else:
```

```
            binary += "0"
    vectors[word] = binary

# Display vectors
print("Word → Binary Vector")
for w in vectors:
    print(w, ":", vectors[w])

# Step 2: Convert binary → decimal
num_vectors = {}
for w in vectors:
    num_vectors[w] = int(vectors[w], 2)

# Step 3: Input query
query = input("\nEnter query (use AND/OR/NOT): ")

# Replace words with numbers
expr = query
for w in words:
    expr = expr.replace(w, str(num_vectors[w]))

# Replace operators
expr = expr.replace("AND", "&").replace("OR", "|").replace("NOT", "~")

# Step 4: Evaluate
result = eval(expr)

# Step 5: Convert result back to binary
result_bin = bin(result).replace("0b","").zfill(len(plays))
```

```
print("\nResult Binary:", result_bin)

# Step 6: Show matching plays
play_names = list(plays.keys())

print("\nMatching Plays:")
for i in range(len(result_bin)):
    if result_bin[i] == "1":
        print(play_names[i])
```

PRAC 2:

RECURSIVE EDIT DISTANCE (Basic)

```
def edit_recursive(s1, s2, m, n):
    if m == 0:
        return n
    if n == 0:
        return m

    if s1[m-1] == s2[n-1]:
        return edit_recursive(s1, s2, m-1, n-1)

    return 1 + min(
        edit_recursive(s1, s2, m, n-1),    # insert
        edit_recursive(s1, s2, m-1, n),    # delete
        edit_recursive(s1, s2, m-1, n-1)   # replace
    )

# Test
s1 = "cat"
s2 = "cut"
print("Recursive:", edit_recursive(s1, s2, len(s1), len(s2)))
```

SIMPLE DP EDIT DISTANCE (BEST ONE)

```
def edit_dp(s1, s2):

    m, n = len(s1), len(s2)

    dp = [[0]*(n+1) for _ in range(m+1)]



    for i in range(m+1):

        dp[i][0] = i

    for j in range(n+1):

        dp[0][j] = j



    for i in range(1, m+1):

        for j in range(1, n+1):



            if s1[i-1] == s2[j-1]:
```

```
            dp[i][j] = dp[i-1][j-1]

        else:

            dp[i][j] = 1 + min(

                dp[i-1][j],     # delete

                dp[i][j-1],     # insert

                dp[i-1][j-1]    # replace

            )



    return dp[m][n]



# Test

print("DP:", edit_dp("cat", "cut"))
```

Weighted edit distance dynamic programming simple one:

```python
def weighted_edit_distance(s1, s2):
    m, n = len(s1), len(s2)

    # 🔥 Different costs
    insert_cost = 1
    delete_cost = 2
    replace_cost = 3

    dp = [[0]*(n+1) for _ in range(m+1)]

    # Base cases
    for i in range(m+1):
        dp[i][0] = i * delete_cost
    for j in range(n+1):
        dp[0][j] = j * insert_cost

    # Fill table
    for i in range(1, m+1):
        for j in range(1, n+1):

            if s1[i-1] == s2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = min(
                    dp[i-1][j] + delete_cost,     # delete
                    dp[i][j-1] + insert_cost,     # insert
                    dp[i-1][j-1] + replace_cost   # replace
                )
```

```
    return dp[m][n]


# 🔥 Test
s1 = input("Enter first string: ")
s2 = input("Enter second string: ")

print("Weighted Edit Distance:", weighted_edit_distance(s1, s2))
```

Word edit distance:

```
def word_edit(s1, s2):
    w1 = s1.split()
    w2 = s2.split()

    m, n = len(w1), len(w2)
    dp = [[0]*(n+1) for _ in range(m+1)]

    for i in range(m+1):
        dp[i][0] = i
    for j in range(n+1):
        dp[0][j] = j
```

```
    for i in range(1, m+1):
        for j in range(1, n+1):

            if w1[i-1] == w2[j-1]:
                dp[i][j] = dp[i-1][j-1]
            else:
                dp[i][j] = 1 + min(
                    dp[i-1][j],
                    dp[i][j-1],
                    dp[i-1][j-1]
                )

    return dp[m][n]

# Test
s1 = "I love coding"
s2 = "I enjoy coding"
print("Word Edit:", word_edit(s1, s2))
```

PRAC3:
Soundex algorithm:

```
def soundex_simple(word):
    codes = {
        "BFPV": "1",
        "CGJKQSXZ": "2",
        "DT": "3",
        "L": "4",
```

```
    "MN": "5",
    "R": "6"
}

word = word.upper()
first = word[0]

result = first
prev = ""

for char in word[1:]:
    for key in codes:
        if char in key:
            code = codes[key]
            break
    else:
        code = ""

    if code != prev and code != "":
        result += code

    prev = code

return (result + "000")[:4]
```

PRAC4:
Ngrams and jaccard similarity

🔥 ✅ **CODE 1**

👉 **N-gram + Bigram + Trigram + Jaccard Similarity + Distance**

```
# ---------- N-GRAM FUNCTION ----------
def ngrams(text, n):
    words = text.lower().split()
    return [tuple(words[i:i+n]) for i in range(len(words)-n+1)]

# ---------- JACCARD ----------
def jaccard_similarity(t1, t2, n):
    A = set(ngrams(t1, n))
    B = set(ngrams(t2, n))
    return len(A & B) / len(A | B)

def jaccard_distance(t1, t2, n):
    return 1 - jaccard_similarity(t1, t2, n)


# ---------- MAIN ----------
text = "this is my first program"

print("Unigram:", ngrams(text, 1))
print("Bigram:", ngrams(text, 2))
print("Trigram:", ngrams(text, 3))

t1 = "this is a book"
t2 = "this is a book of english"

print("\nJaccard Similarity:", jaccard_similarity(t1, t2, 2))
print("Jaccard Distance:", jaccard_distance(t1, t2, 2))
```

🔥 ✅ CODE 2

👉 **Manual Jaccard (NO inbuilt set operations)**

```
# ---------- N-GRAM ----------
def ngrams(text, n):
    words = text.lower().split()
    result = []
    for i in range(len(words)-n+1):
        result.append(tuple(words[i:i+n]))
    return result


# ---------- MANUAL JACCARD ----------
def jaccard_manual(t1, t2, n):
    A = ngrams(t1, n)
    B = ngrams(t2, n)


    # Intersection
    common = []
    for x in A:
        if x in B and x not in common:
            common.append(x)
```

```
    # Union
    union = []
    for x in A + B:
        if x not in union:
            union.append(x)

    return len(common) / len(union)


# ---------- MAIN ----------
t1 = "new york city is big"
t2 = "new york city is nice"

print("Manual Jaccard:", jaccard_manual(t1, t2, 2))
```

PRAC5:

Page rank

```
graph = {
    "A": ["B", "C"],
```

```
    "B": ["C"],
    "C": ["A"]
}

d = 0.85

# Initial rank
pr = {p: 1 for p in graph}

for _ in range(3):   # iterations
    new_pr = {}

    for page in graph:
        total = 0

        for other in graph:
            if page in graph[other]:   # if link exists
                total += pr[other] / len(graph[other])

        new_pr[page] = (1 - d) + d * total

    pr = new_pr

print(pr)
```

Prac 6 : Document similarity 👍

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
```

```
import numpy as np

nltk.download('punkt')
nltk.download('stopwords')

def process(text):
    words = word_tokenize(text.lower())
    words = [w for w in words if w.isalpha()]      # keep only words
    words = [w for w in words if w not in stopwords.words('english')]

    freq = {}
    for w in words:
        freq[w] = freq.get(w, 0) + 1
    return freq


def similarity(d1, d2):
    all_words = list(set(d1) | set(d2))   # combine all words

    v1 = [d1.get(w, 0) for w in all_words]
    v2 = [d2.get(w, 0) for w in all_words]

    return np.dot(v1, v2) / (np.linalg.norm(v1) * np.linalg.norm(v2))


# MAIN
text1 = open("text1.txt").read()
text2 = open("text2.txt").read()

d1 = process(text1)
```

```
d2 = process(text2)

print("Similarity:", similarity(d1, d2))
```

ALTERNATE CODE:

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import numpy as np

nltk.download('punkt')
nltk.download('stopwords')

# -------- PROCESS FUNCTION --------
def process(text):
    words = word_tokenize(text.lower())

    # remove punctuation
    words = [w for w in words if w.isalpha()]

    # remove stopwords
    words = [w for w in words if w not in stopwords.words('english')]

    # count frequency
    freq = {}
    for w in words:
        if w in freq:
            freq[w] += 1
        else:
```

```
        freq[w] = 1

    return freq


# -------- SIMILARITY FUNCTION --------
def similarity(d1, d2):

    # Step 1: union of words
    all_words = list(set(d1) | set(d2))

    # Step 2: create vector v1 (simple if-else)
    v1 = []
    for w in all_words:
        if w in d1:
            v1.append(d1[w])
        else:
            v1.append(0)

    # Step 3: create vector v2 (simple if-else)
    v2 = []
    for w in all_words:
        if w in d2:
            v2.append(d2[w])
        else:
            v2.append(0)

    # Step 4: cosine similarity
    dot = np.dot(v1, v2)
    norm1 = np.linalg.norm(v1)
```

```
    norm2 = np.linalg.norm(v2)

    return dot / (norm1 * norm2)


# -------- MAIN --------
text1 = open("text1.txt").read()
text2 = open("text2.txt").read()

d1 = process(text1)
d2 = process(text2)

print("Similarity:", similarity(d1, d2))
```

Prac 7 : TEXT PROCESSING *STOPWARDS AND TOKENIZATION*

(a) Direct Text – Stopword Removal

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

# Download once
nltk.download('punkt')
nltk.download('stopwords')
```

```
# Input sentence
text = "This is a sample sentence, showing stop word removal."

# Get stopwords
stop_words = set(stopwords.words('english'))

# Tokenize
words = word_tokenize(text)

# Remove stopwords
filtered = [w for w in words if w.lower() not in stop_words]

# Output
print("Original:", words)
print("Filtered:", filtered)
```

(b) File Input → File Output (COMPLETE FIXED CODE)

```
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize

nltk.download('punkt')
nltk.download('stopwords')

# Load stopwords
stop_words = set(stopwords.words('english'))

# Read from file
with open("input.txt", "r") as f:
```

```
    text = f.read()

# Tokenize
words = word_tokenize(text)

# Remove stopwords
filtered = [w for w in words if w.lower() not in stop_words]

# Write to new file
with open("output.txt", "w") as f:
    f.write(" ".join(filtered))

# Print result
print("Filtered words:", filtered)
```

PRACTICAL 8 :CRAWLER

```
import requests
from bs4 import BeautifulSoup
import re

def crawler(url, word):
    # 1. Fetch page
    response = requests.get(url)
    if response.status_code != 200:
        print("Failed to load page")
        return

    # 2. Parse HTML
```

```
    soup = BeautifulSoup(response.text, "html.parser")

    # 3. Remove script/style
    for tag in soup(["script", "style"]):
        tag.extract()

    # 4. Get text
    text = soup.get_text()

    # 5. Search word
    matches = re.findall(r'\b' + word + r'\b', text, re.IGNORECASE)
    print(f"\n'{word}' found {len(matches)} times")

    # 6. Extract links
    links = soup.find_all("a")
    print(f"\nTotal links found: {len(links)}")

    print("\nFirst 10 links:")
    for i, link in enumerate(links[:10], 1):
        print(i, link.get("href"))


# --- MAIN ---
url = input("Enter URL: ")
word = input("Enter word to search: ")

if not url.startswith("http"):
    url = "https://" + url

crawler(url, word)
```

PRACTICAL 9 : *INVERTED INDEX AND RETRIEVAL*

```
import string

# Step 1: Preprocess text
def preprocess(text):
    text = text.lower()
    text = text.translate(str.maketrans("", "", string.punctuation))
    return text.split()

# Step 2: Build inverted index
def build_index(docs):
    index = {}

    for doc_id, text in docs.items():
        words = preprocess(text)

        for word in words:
            if word not in index:
                index[word] = set()
            index[word].add(doc_id)

    return index

# Step 3: Search (Retrieval)
def search(index, query):
    terms = preprocess(query)

    if not terms:
```

```
        return set()

    if terms[0] in index:
        result = index[terms[0]].copy()
    else:
        return set()

    for term in terms[1:]:
        if term in index:
            result = result.intersection(index[term])
        else:
            return set()

    return result


# ---------------- MAIN ----------------

documents = {
    1: "Information retrieval is essential for search.",
    2: "Search engines use algorithms.",
    3: "Retrieval of information is key.",
    4: "Algorithms improve search engines."
}

# Build index
index = build_index(documents)

# Display index in alphabetical order
print("\n--- Inverted Index (Alphabetical) ---")
for word in sorted(index):
    print(word, "->", index[word])

# Count unique terms
print("\nTotal unique terms:", len(index))

# Retrieval
```

```
query = input("\nEnter query: ")
result = search(index, query)

print("\n--- Search Results ---")
print("Found in documents:", result)
```

PRAC 10 : XML PARSING XML → CSV ONLY)

```
import requests
import xml.etree.ElementTree as ET
import csv

# Step 1: Load RSS (XML from URL)
def load_xml(url, filename):
    response = requests.get(url)
    with open(filename, "wb") as f:
        f.write(response.content)
    print("XML file saved")

# Step 2: Parse XML
def parse_xml(filename):
    tree = ET.parse(filename)
    root = tree.getroot()

    news_list = []

    for item in root.findall(".//item"):
        data = {
            "title": item.findtext("title"),
            "link": item.findtext("link"),
            "description": item.findtext("description"),
```

```
            "pubDate": item.findtext("pubDate")
        }
        news_list.append(data)

    return news_list

# Step 3: Save to CSV
def save_csv(data, filename):
    fields = ["title", "link", "description", "pubDate"]

    with open(filename, "w", newline='', encoding="utf-8") as f:
        writer = csv.DictWriter(f, fieldnames=fields)
        writer.writeheader()
        writer.writerows(data)

    print("CSV file saved")

# -------- MAIN --------
url = "https://feeds.feedburner.com/50WordStories"

xml_file = "data.xml"
csv_file = "data.csv"

load_xml(url, xml_file)
data = parse_xml(xml_file)
save_csv(data, csv_file)
```